# Terraform Collaboration Guide

Software Architecture

Assignment Guide

CSSE6400 Teaching Team

> Aside
>
> Use this guide when setting up Terraform for your team assignment. Each team should use one shared Terraform state file stored in S3.

## 1 Overview

Terraform records the resources it manages in a state file. When working alone, this is usually a local file named `terraform.tfstate`. When working in a team, local state causes problems because each person may have a different view of the infrastructure.

For the assignment, your team should store Terraform state in a shared S3 bucket. This lets every team member run `terraform plan` against the same state. It also allows Terraform to lock the state while someone is applying changes.

> Warning
>
> Never commit `terraform.tfstate`, `terraform.tfstate.backup`, `.terraform/`, AWS credentials, or Learner Lab tokens.

## 2 Choose an Account, Region, and Bucket Name

Your team needs one S3 bucket for Terraform state. Choose one AWS account, one AWS region, and one bucket name before configuring Terraform. Every team member must use the ***same*** account, region, bucket, and state file path.

```
AWS account ID: 123456789012
AWS region: us-east-1
S3 state bucket: csse6400-t00-123456789012-us-east-1-tfstate
State file path: project-name/terraform.tfstate
```

Replace the following values.

- `123456789012` with the ID of the shared AWS account allocated to your team for the project.
- `us-east-1` with the AWS region your team is using.
- `t00` with your team number.
- `project-name` with the name of your project.

> Info
>
> S3 bucket names are globally unique. Including the team number, AWS account ID, and AWS region makes collisions unlikely and helps your team spot accidental account or region mismatches.

## 3 Step 1: Create the Bucket

One team member should create the bucket. Do this ***before*** running `terraform init` with the S3 backend. First set your chosen values in the terminal.

```
export AWS_REGION=us-east-1
export TEAM_NUMBER=00
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export TF_STATE_BUCKET="csse6400-t${TEAM_NUMBER}-${AWS_ACCOUNT_ID}-${AWS_REGION}-
    tfstate"
```

> ⚠ Warning
>
> Use the shared AWS account allocated to your team to host the project. Do ***not*** create a separate state bucket in each team member's personal Learner Lab account.

For `us-east-1`, use:

```
aws s3api create-bucket \
  --bucket "$TF_STATE_BUCKET" \
  --region "$AWS_REGION"

aws s3api put-bucket-versioning \
  --bucket "$TF_STATE_BUCKET" \
  --versioning-configuration Status=Enabled
```

For other regions, set `AWS_REGION` before generating the bucket name, and include a location constraint:

```
export AWS_REGION=ap-southeast-2
export TF_STATE_BUCKET="csse6400-t${TEAM_NUMBER}-${AWS_ACCOUNT_ID}-${AWS_REGION}-
    tfstate"

aws s3api create-bucket \
  --bucket "$TF_STATE_BUCKET" \
  --region "$AWS_REGION" \
  --create-bucket-configuration LocationConstraint="$AWS_REGION"

aws s3api put-bucket-versioning \
  --bucket "$TF_STATE_BUCKET" \
  --versioning-configuration Status=Enabled
```

> ℹ Info
>
> Bucket versioning is recommended so that it is easier to recover from accidental state changes.

> **Warning**
>
> The bucket that stores Terraform state should be created ***before*** the backend is configured. Do ***not*** put this same bucket in the Terraform configuration that will use it as a backend.

## 4 Step 2: Create Backend Files

In your Terraform directory, create `backend.tf`.

```
 » cat backend.tf

terraform {
  backend "s3" {}
}
```

Create `backend.hcl`. This file tells Terraform which S3 bucket should store the shared state.

```
 » cat backend.hcl

bucket = "csse6400-t00-123456789012-us-east-1-tfstate"
key = "project-name/terraform.tfstate"
region = "us-east-1"
encrypt = true
use_lockfile = true
```

Replace `project-name` with the name of your project. The `region` value must match the region where your team created the bucket.

> **Info**
>
> `use_lockfile = true` enables S3 state locking. This helps stop two team members from applying changes to the same state at the same time.

> **Warning**
>
> Backend blocks cannot use Terraform variables. Do not write `bucket = var.bucket_name` inside a backend configuration. If Terraform rejects `use_lockfile`, update Terraform or ask a tutor.

## 5 Step 3: Initialise Terraform

Run Terraform initialisation from the Terraform directory.

```
$ terraform init -backend-config=backend.hcl
```

If this is a fresh project, Terraform should initialise the S3 backend. If there is already local state, Terraform may ask whether to migrate that state to S3.

> ⚠ Warning
>
> Only migrate state from the machine that has the correct current state. If multiple people have already run `terraform apply` locally, stop and ask a tutor before migrating.

# 6 Step 4: Ignore Local Files

Make sure generated Terraform files and local credentials are included in your `.gitignore` file.

```
cat .gitignore
.terraform/
terraform.tfstate
terraform.tfstate.backup
*.tfstate
*.tfstate.backup
credentials
aws.env
```

Commit `backend.tf`. You may commit `backend.hcl`, if it only contains the team bucket name and no secrets. If each student needs different local settings, commit `backend.example.hcl` instead, and add `backend.hcl` to `.gitignore`.

# 7 Normal Team Workflow

Before making changes:

```
git pull
terraform init -backend-config=backend.hcl
terraform validate
terraform plan
```

Before opening a pull request:

```
terraform fmt
terraform validate
terraform plan
```

When applying shared infrastructure changes:

```
git pull
aws sts get-caller-identity
terraform init -backend-config=backend.hcl
terraform plan
terraform apply
```

> **Warning**
>
> Only one person should run `terraform apply` at a time. Tell your team before applying. Do not use `-lock=false` during normal team work.

# 8 Changing the Backend

If you change `backend.tf` or `backend.hcl`, reinitialise Terraform. Do this carefully: a wrong bucket name or wrong region can point your team at a different state file.

```
$ terraform init -reconfigure -backend-config=backend.hcl
```

> **Warning**
>
> If `terraform plan` wants to destroy something important, do not apply until the team understands why. Ask a tutor if unsure.